

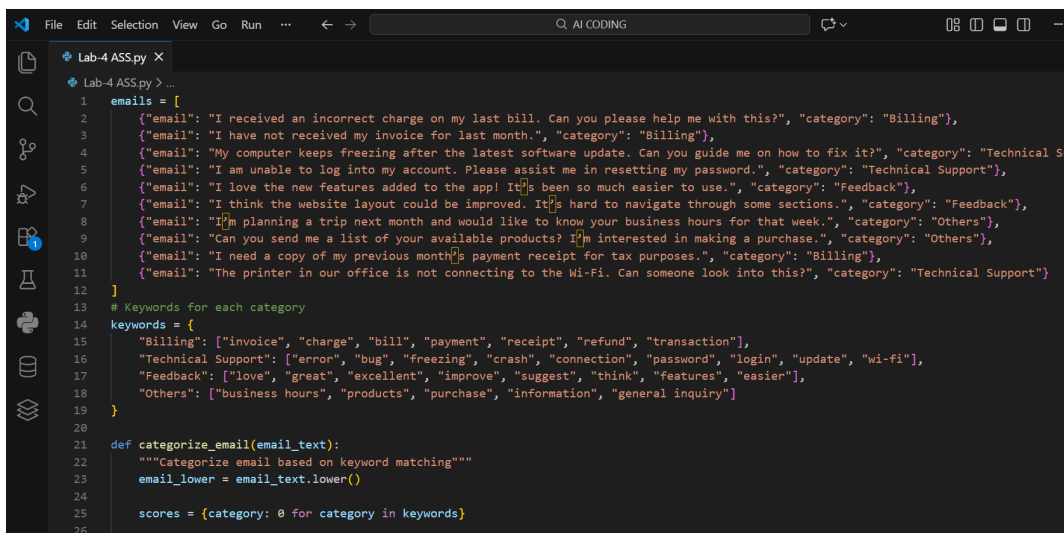
Lab 4: Advanced Prompt Engineering: Zero-shot, one-shot, and few-shot techniques

Objective: To explore and compare Zero-shot, One-shot, and Few-shot prompting techniques for classifying emails into predefined categories using a large language model (LLM).

1. Suppose that you work for a company that receives hundreds of customer emails daily. Management wants to automatically classify emails into categories like "Billing", "Technical Support", "Feedback", and "Others" before assigning them to appropriate departments. Instead of training a new model, your task is to use prompt engineering techniques with an existing LLM to handle the classification.

PROMPT : "Classify the following email into one of these four categories: Billing, Technical Support, Feedback, or Others, based on the content of the message.

Email: 'I have not received my invoice for last month.'"



```

File Edit Selection View Go Run ... AI CODING
Lab-4 ASS.py x
Lab-4 ASS.py > ...
1 emails = [
2     {"email": "I received an incorrect charge on my last bill. Can you please help me with this?", "category": "Billing"},
3     {"email": "I have not received my invoice for last month.", "category": "Billing"},
4     {"email": "My computer keeps freezing after the latest software update. Can you guide me on how to fix it?", "category": "Technical Support"},
5     {"email": "I am unable to log into my account. Please assist me in resetting my password.", "category": "Technical Support"},
6     {"email": "I love the new features added to the app! It's been so much easier to use.", "category": "Feedback"},
7     {"email": "I think the website layout could be improved. It's hard to navigate through some sections.", "category": "Feedback"},
8     {"email": "I'm planning a trip next month and would like to know your business hours for that week.", "category": "Others"},
9     {"email": "Can you send me a list of your available products? I'm interested in making a purchase.", "category": "Others"},
10    {"email": "I need a copy of my previous month's payment receipt for tax purposes.", "category": "Billing"},
11    {"email": "The printer in our office is not connecting to the Wi-Fi. Can someone look into this?", "category": "Technical Support"}
12 ]
13 # Keywords for each category
14 keywords = {
15     "Billing": ["invoice", "change", "bill", "payment", "receipt", "refund", "transaction"],
16     "Technical Support": ["error", "bug", "freezing", "crash", "connection", "password", "login", "update", "wi-fi"],
17     "Feedback": ["love", "great", "excellent", "improve", "suggest", "think", "features", "easier"],
18     "Others": ["business hours", "products", "purchase", "information", "general inquiry"]
19 }
20
21 def categorize_email(email_text):
22     """Categorize email based on keyword matching"""
23     email_lower = email_text.lower()
24
25     scores = {category: 0 for category in keywords}
26
  
```

```

File Edit Selection View Go Run ... < -> Q AI CODING
Lab-4 ASS.py X
21 def categorize_email(email_text):
22     # Categorize email based on keyword matching
23     email_lower = email_text.lower()
24
25     scores = {category: 0 for category in keywords}
26
27     for category, words in keywords.items():
28         for word in words:
29             if word.lower() in email_lower:
30                 scores[category] += 1
31
32     # Return the category with highest score, default to "Others"
33     best_category = max(scores, key=scores.get)
34     return best_category if scores[best_category] > 0 else "Others"
35
36 # Test the categorization
37 print("Email Categorization Results:\n")
38 for email_data in emails:
39     email_text = email_data["email"]
40     actual_category = email_data["category"]
41     predicted_category = categorize_email(email_text)
42     match = "✓" if predicted_category == actual_category else "X"
43
44     print(f"{match} Email: {email_text[:60]}...")
45     print(f"   Actual: {actual_category}, Predicted: {predicted_category}\n")
46

```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + -
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-4 ASS.py"
Email Categorization Results:
✓ Email: I received an incorrect charge on my last bill. Can you plea...
Actual: Billing, Predicted: Billing
✓ Email: I have not received my invoice for last month....
Actual: Billing, Predicted: Billing
✓ Email: My computer keeps freezing after the latest software update....
Actual: Technical Support, Predicted: Technical Support
✓ Email: I am unable to log into my account. Please assist me in rese...
Actual: Technical Support, Predicted: Technical Support
✓ Email: I love the new features added to the app! It's been so much ...
Actual: Feedback, Predicted: Feedback
✓ Email: I think the website layout could be improved. It's hard to n...
Actual: Feedback, Predicted: Feedback
✓ Email: I'm planning a trip next month and would like to know your b...
Actual: Others, Predicted: Others
✓ Email: Can you send me a list of your available products? I'm inter...
Actual: Others, Predicted: Others
✓ Email: I need a copy of my previous month's payment receipt for tax...

```

b. Zero-shot Prompting:

- Design a prompt that asks the LLM to classify a single email without providing any examples.
- Example prompt:

"Classify the following email into one of the following categories: Billing, Technical Support, Feedback, Others. Email: 'I have not received my invoice for last month.'"

```

47
48 #Task -2
49 # Test email categorization with new example
50 test_email = "I need assistance with resetting my account password."
51 predicted = categorize_email(test_email)
52 print(f"Test Email: {test_email}")
53 print(f"Predicted Category: {predicted}")
54

```

```

✓ Email: I need a copy of my previous month's payment receipt for tax...
Actual: Billing, Predicted: Billing
✓ Email: The printer in our office is not connecting to the Wi-Fi. Ca...
Actual: Technical Support, Predicted: Technical Support
Test Email: I need assistance with resetting my account password.
Predicted Category: Technical Support

```

C. One-shot Prompting:

- Add one labeled example before asking the model to classify a new email.

```
53 print(f'Predicted Category: {predicted} ')
54
55 #Task-3
56 # Task-3: Classify new email
57 task3_email = "I can't find where to update my subscription plan. Can you guide me?"
58 predicted = categorize_email(task3_email)
59 print(f"\nTask-3 Email: {task3_email}")
60 print(f"Predicted Category: {predicted}")
61
62 #Task-4
```

```
✓ Email: The printer in our office is not connecting to the Wi-Fi. Ca...
Actual: Technical Support, Predicted: Technical Support

Test Email: I need assistance with resetting my account password.
Predicted Category: Technical Support

Task-3 Email: I can't find where to update my subscription plan. Can you guide me?
Predicted Category: Technical Support
```

Prompt : "Here's an example email and its category:

'I love your product and the new features, keep up the great work!' → Feedback

Now, classify the following email:

'I'm having trouble accessing my order history on the website.'"

d. Few-shot Prompting:

- Use 3–5 labeled examples in your prompt before asking the model to classify a new email.

```
#Task-4:
# Test Email Data
test_emails = [
    "I can't access my order history on the website, can you help me?",
    "I think your product is great, but the packaging could be better.",
    "I was charged twice for my last purchase, how can I get a refund?",
    "I need help resetting my password. I keep getting an error.",
    "How do I unsubscribe from your newsletter?"
]
```

```
PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" c:/Users/HP/Desktop/AI/ASS-4.5.py
PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" c:/Users/HP/Desktop/AI/ASS-4.5.py
Email Categorization Results:

✓ Email: I received an incorrect charge on my last bill. Can you plea...
Actual: Billing, Predicted: Billing

✓ Email: I have not received my invoice for last month...
Actual: Billing, Predicted: Billing

✓ Email: My computer keeps freezing after the latest software update....
Actual: Technical Support, Predicted: Technical Support

✓ Email: I am unable to log into my account. Please assist me in rese...
Actual: Technical Support, Predicted: Technical Support

✓ Email: I love the new features added to the app! It's been so much ...
Actual: Feedback, Predicted: Feedback

✓ Email: I think the website layout could be improved. It's hard to n...
Actual: Feedback, Predicted: Feedback

✓ Email: I'm planning a trip next month and would like to know your b...
Actual: Others, Predicted: Others

✓ Email: Can you send me a list of your available products? I'm inter...
Actual: Others, Predicted: Others

✓ Email: I need a copy of my previous month's payment receipt for tax...
Actual: Billing, Predicted: Billing

✓ Email: The printer in our office is not connecting to the Wi-Fi. Ca...
Actual: Technical Support, Predicted: Technical Support

Test Email: I need assistance with resetting my account password.
Predicted Category: Technical Support

Task-3 Email: I can't find where to update my subscription plan. Can you guide me?
Predicted Category: Technical Support
PS C:\Users\HP\Desktop\AI>
```

TASK 2 :

Travel Query Classification

Scenario:

A travel assistant must classify queries into Flight Booking, Hotel Booking, Cancellation, or General Travel Info.

Tasks:

- Prepare labeled travel queries.

```
File Edit Selection View Go Run ... Q AI CODING
Lab-4 ASS.py X
Lab-4 ASS.py > ...

70 }
71
72 # A. Labeled Travel Queries
73
74 travel_queries = [
75     {"query": "I want to book a flight from Hyderabad to Delhi.", "category": "Flight Booking"},
76     {"query": "Please book a round-trip flight to Mumbai for next week.", "category": "Flight Booking"},
77
78     {"query": "I need a hotel room in Goa for three nights.", "category": "Hotel Booking"},
79     {"query": "Can you reserve a budget hotel near the airport?", "category": "Hotel Booking"},
80
81     {"query": "I want to cancel my flight scheduled for tomorrow.", "category": "Cancellation"},
82     {"query": "Can I get a refund if I cancel my ticket?", "category": "Cancellation"},
83
84     {"query": "What documents are required for international travel?", "category": "General Travel Info"},
85     {"query": "Are meals included in economy class flights?", "category": "General Travel Info"},
86
87     {"query": "Is travel insurance mandatory for domestic flights?", "category": "General Travel Info"},
88     {"query": "Please cancel my hotel booking for this weekend.", "category": "Cancellation"}
89 ]
90 # b. Classify travel query
```

- Apply Zero-shot prompting.

```
88     {"query": "Please cancel my hotel booking for this weekend.", "category": "Cancellation"}
89 ]
90 # b. Classify travel query
91 travel_query = "I want to cancel my flight scheduled for tomorrow."
92 predicted_travel_category = categorize_email(travel_query)
93 print(f"\nTravel Query: {travel_query}")
94 print(f"Predicted Category: {predicted_travel_category}")
95
```

- Apply One-shot prompting.

```
94 print(f"Predicted Category: {predicted_travel_category}")
95
96 # C. Classify the given travel query
97 query_to_classify = "I need a hotel room in Goa for three nights."
98 predicted_category = categorize_email(query_to_classify)
99 print(f"\nQuery: {query_to_classify}")
100 print(f"Category: {predicted_category}")
101
```

- Apply Few-shot prompting.

```
100 print(f"Category: {predicted_category}")
101
102 # D. Classify the given travel query
103 query_to_classify = "Please cancel my hotel booking."
104 predicted_category = categorize_email(query_to_classify)
105 print(f"\nQuery: {query_to_classify}")
106 print(f"Category: {predicted_category}")
107
```

e. Comparison of Zero-shot, One-shot, and Few-shot Prompting

- **Zero-shot prompting** classifies travel queries without any examples. It works for simple and clear queries but may give inconsistent results for ambiguous or complex queries.
- **One-shot prompting** provides one example before classification. This improves accuracy compared to zero-shot prompting, as the model gets a reference for how categories are assigned.
- **Few-shot prompting** uses multiple labeled examples. This method produces the **most consistent and accurate results** because the model clearly understands the pattern and differences between categories.

TASK 3:

Programming Question Type Identification

Scenario:

A coding help chatbot must classify queries into Syntax Error, Logic Error, Optimization, or Conceptual Question.

a. Prepare coding-related user queries.

PROMPT: Generate 6 programming-related user queries.

Each query should belong to one category:

Syntax Error, Logic Error, Optimization, or Conceptual Question.

Label each query with its category.

```
107
108 # 3.Labeled Programming Queries
109
110 programming_queries = [
111     {"query": "I am getting a syntax error near ';' in my Python code.", "category": "Syntax Error"},
112     {"query": "Why does my loop run infinitely even when the condition seems correct?", "category": "Logic Error"},
113     {"query": "How can I reduce the time complexity of this sorting algorithm?", "category": "Optimization"},
114     {"query": "What is the difference between a list and a tuple in Python?", "category": "Conceptual Question"},
115
116     {"query": "The compiler says unexpected token '{' in my program.", "category": "Syntax Error"},
117     {"query": "My program runs but gives the wrong output.", "category": "Logic Error"},
118     {"query": "How can I optimize this code to use less memory?", "category": "Optimization"},
119     {"query": "Can you explain what recursion is with an example?", "category": "Conceptual Question"}
120 ]
121
```

b. Perform Zero-shot classification.

PROMPT: Classify the following programming question as:

Syntax Error, Logic Error, Optimization, or Conceptual Question.

```
121
122 # 8. keywords for programming query categorization
123 programming_keywords = {
124     "Syntax Error": ["syntax error", "unexpected token", "compile error", "invalid syntax"],
125     "Logic Error": ["wrong output", "infinitely", "incorrect result", "logic error", "bug"],
126     "Optimization": ["optimize", "reduce", "time complexity", "memory", "performance", "efficient"],
127     "Conceptual Question": ["what is", "explain", "difference between", "how does", "definition"]
128 }
129
130 def categorize_programming_query(query_text):
131     """Categorize programming query based on keyword matching"""
132     query_lower = query_text.lower()
133
134     scores = {category: 0 for category in programming_keywords}
135
136     for category, words in programming_keywords.items():
137         for word in words:
138             if word.lower() in query_lower:
139                 scores[category] += 1
140
141     best_category = max(scores, key=scores.get)
142     return best_category if scores[best_category] > 0 else "Conceptual Question"
143
```

c. Perform One-shot classification.

PROMPT : Example:

Question: "Why am I getting a syntax error in Python?"

Category: Syntax Error Now classify this question as:

Syntax Error, Logic Error, Optimization, or Conceptual Question.

```
142 best_category = max(scores, key=scores.get)
143 return best_category if scores[best_category] > 0 else "Conceptual Question"
144
145 # Classify the given programming query
146 programming_query_to_classify = "My program runs but gives the wrong output."
147 predicted_programming_category = categorize_programming_query(programming_query_to_classify)
148 print(f"\nProgramming Query: {programming_query_to_classify}")
149 print(f"Category: {predicted_programming_category}")
150
```

d. Perform Few-shot classification.

PROMPT:

Examples:

Question: "Missing semicolon in C code"

Category: Syntax Error

Now classify the following question:

"<paste query here>"

Choose one category:

Syntax Error, Logic Error, Optimization, Conceptual Question

```
147 print(f"\nProgramming Query: {programming_query_to_classify}")
148 print(f"Category: {predicted_programming_category}")
149
150 # Classify the given programming query
151 query = "<paste query here>"
152 predicted_category = categorize_programming_query(query)
153 print(f"\nQuestion: {query}")
154 print(f"Category: {predicted_category}")
```

```
147 print(f"\nProgramming Query: {programming_query_to_classify}")
148 print(f"Category: {predicted_programming_category}")
149
150 # Classify the given programming query
151 query = "<paste query here>"
152 predicted_category = categorize_programming_query(query)
153 print(f"\nQuestion: {query}")
154 print(f"Category: {predicted_category}")

PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/
/Lab-4 ASS.py"
Predicted Category: Technical Support
Task-3 Email: I can't find where to update my subscription plan. Can you guide me?
Predicted Category: Technical Support
Travel Query: I want to cancel my flight scheduled for tomorrow.
Predicted Category: Others
Query: I need a hotel room in Goa for three nights.
Category: Others
Query: Please cancel my hotel booking.
Category: Others
Programming Query: My program runs but gives the wrong output.
Category: Logic Error
Question: <paste query here>
Category: Conceptual Question
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
```

e. Analyze improvements in technical accuracy.

Compare Zero-shot, One-shot, and Few-shot prompting for classifying programming questions. State which method is most accurate and why.

Comparison of Prompting Methods :

- **Zero-shot prompting:**

The model classifies questions without examples. It works reasonably well but may misclassify ambiguous queries due to lack of context.

- **One-shot prompting:**

Providing one example improves understanding of the task, leading to better accuracy than zero-shot, but errors can still occur for varied question types.

- **Few-shot prompting:**

Multiple examples clearly define each category, helping the model recognize patterns more accurately and consistently.

Most Accurate Method:

Few-shot prompting is the most accurate because multiple labeled examples guide the model's reasoning, reduce ambiguity, and improve consistency across different programming question types.

TASK-4 : Social Media Post Categorization Scenario: A social media analytics tool must classify posts into Promotion, Complaint, Appreciation, or Inquiry.

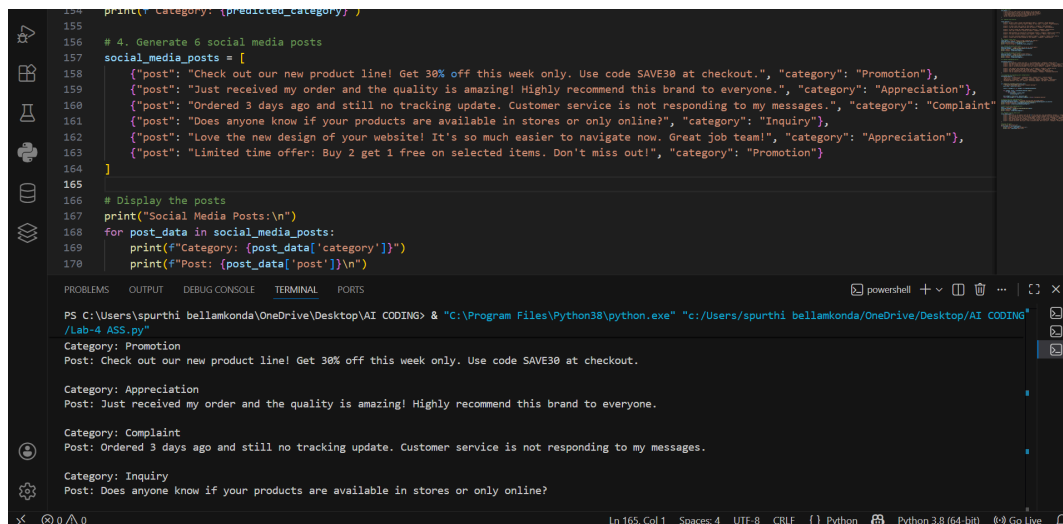
1. Prepare sample social media posts.

PROMPT: Generate 6 social media posts.

Each post should belong to one category:

Promotion, Complaint, Appreciation, or Inquiry.

Label each post with its category.



```
154 print(f'Category: {predicted_category}')
```

```
155
```

```
156 # 4. Generate 6 social media posts
```

```
157 social_media_posts = [
```

```
158     ("post": "Check out our new product line! Get 30% off this week only. Use code SAVE30 at checkout.", "category": "Promotion"),
```

```
159     ("post": "Just received my order and the quality is amazing! Highly recommend this brand to everyone.", "category": "Appreciation"),
```

```
160     ("post": "Ordered 3 days ago and still no tracking update. Customer service is not responding to my messages.", "category": "Complaint"),
```

```
161     ("post": "Does anyone know if your products are available in stores or only online?", "category": "Inquiry"),
```

```
162     ("post": "Love the new design of your website! It's so much easier to navigate now. Great job team!", "category": "Appreciation"),
```

```
163     ("post": "Limited time offer: Buy 2 get 1 free on selected items. Don't miss out!", "category": "Promotion")
```

```
164 ]
```

```
165
```

```
166 # Display the posts
```

```
167 print("Social Media Posts:\n")
```

```
168 for post_data in social_media_posts:
```

```
169     print(f'Category: {post_data["category"]}')
170     print(f'Post: {post_data["post"]}\n')
```

Category: Promotion
Post: Check out our new product line! Get 30% off this week only. Use code SAVE30 at checkout.

Category: Appreciation
Post: Just received my order and the quality is amazing! Highly recommend this brand to everyone.

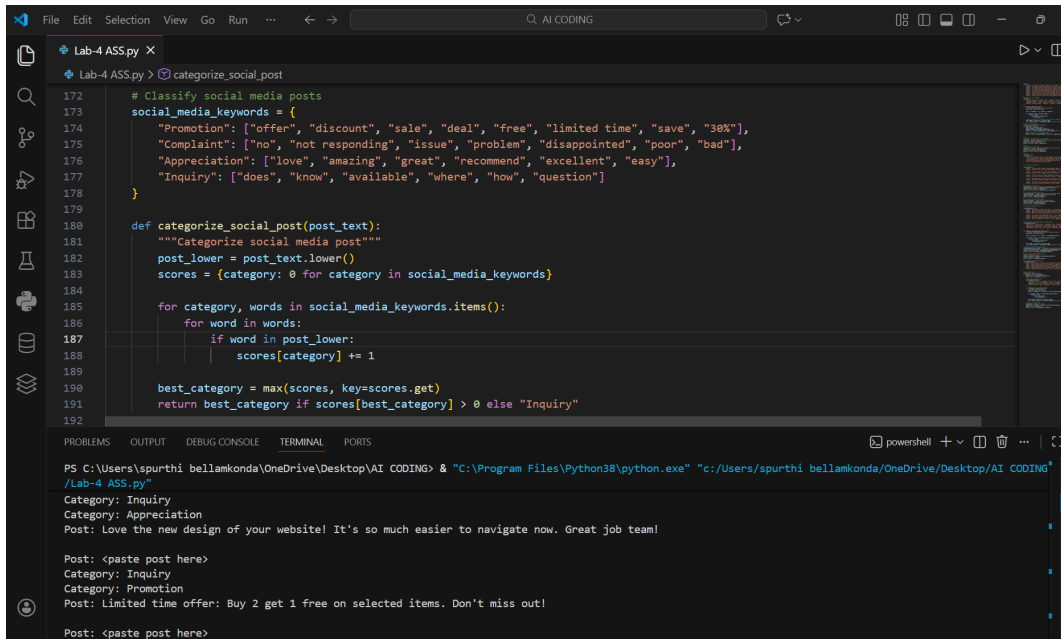
Category: Complaint
Post: Ordered 3 days ago and still no tracking update. Customer service is not responding to my messages.

Category: Inquiry
Post: Does anyone know if your products are available in stores or only online?

2. Use Zero-shot prompting.

PROMPT :

Classify the following social media post as:
Promotion, Complaint, Appreciation, or Inquiry.



```
172 # Classify social media posts
173 social_media_keywords = {
174     "Promotion": ["offer", "discount", "sale", "deal", "free", "limited time", "save", "30%"],
175     "Complaint": ["no", "not responding", "issue", "problem", "disappointed", "poor", "bad"],
176     "Appreciation": ["love", "amazing", "great", "recommend", "excellent", "easy"],
177     "Inquiry": ["does", "know", "available", "where", "how", "question"]
178 }
179
180 def categorize_social_post(post_text):
181     """Categorize social media post"""
182     post_lower = post_text.lower()
183     scores = {category: 0 for category in social_media_keywords}
184
185     for category, words in social_media_keywords.items():
186         for word in words:
187             if word in post_lower:
188                 scores[category] += 1
189
190     best_category = max(scores, key=scores.get)
191     return best_category if scores[best_category] > 0 else "Inquiry"
192
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING"/Lab-4 ASS.py

Category: Inquiry
Category: Appreciation
Post: Love the new design of your website! It's so much easier to navigate now. Great job team!

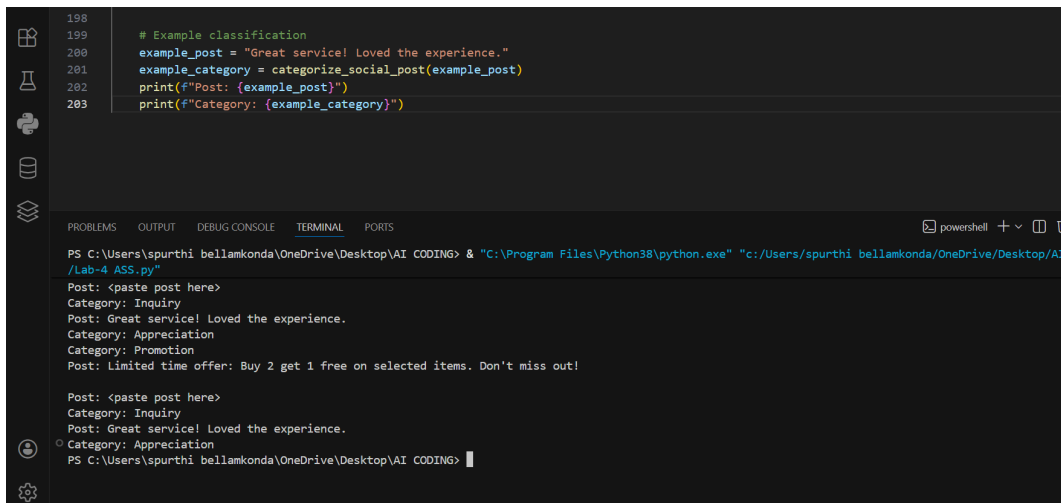
Post: <paste post here>
Category: Inquiry
Category: Promotion
Post: Limited time offer: Buy 2 get 1 free on selected items. Don't miss out!

Post: <paste post here>

3. Use One-shot prompting.

PROMPT: Post: "Great service! Loved the experience."
Category: Appreciation

Now classify the following post as:
Promotion, Complaint, Appreciation, or Inquiry.



```
198
199 # Example classification
200 example_post = "Great service! Loved the experience."
201 example_category = categorize_social_post(example_post)
202 print(f"Post: {example_post}")
203 print(f"Category: {example_category}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING"/Lab-4 ASS.py

Post: <paste post here>
Category: Inquiry
Post: Great service! Loved the experience.
Category: Appreciation
Category: Promotion
Post: Limited time offer: Buy 2 get 1 free on selected items. Don't miss out!

Post: <paste post here>
Category: Inquiry
Post: Great service! Loved the experience.
Category: Appreciation
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>

4. Use Few-shot prompting.

PROMPT: Post: "Buy one get one free! Limited offer."
Category: Promotion

Post: "Very disappointed with the delivery delay."

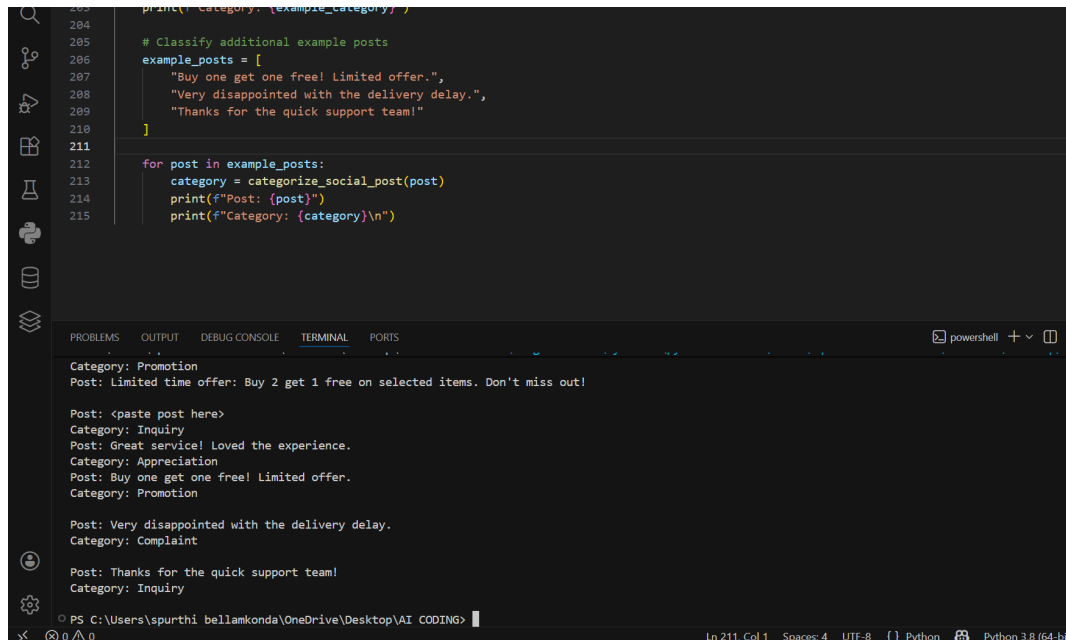
Category: Complaint

Post: "Thanks for the quick support team!"

Category: Appreciation

Choose one category:

Promotion, Complaint, Appreciation, Inquiry



```
203 print(f'Category: {example_category}')
```

```
204
```

```
205 # Classify additional example posts
```

```
206 example_posts = [
```

```
207     "Buy one get one free! Limited offer.",
```

```
208     "Very disappointed with the delivery delay.",
```

```
209     "Thanks for the quick support team!"
```

```
210 ]
```

```
211
```

```
212 for post in example_posts:
```

```
213     category = categorize_social_post(post)
```

```
214     print(f"Post: {post}")
```

```
215     print(f"Category: {category}\n")
```

Category: Promotion
Post: Limited time offer: Buy 2 get 1 free on selected items. Don't miss out!

Post: <paste post here>
Category: Inquiry
Post: Great service! Loved the experience.
Category: Appreciation
Post: Buy one get one free! Limited offer.
Category: Promotion

Post: Very disappointed with the delivery delay.
Category: Complaint

Post: Thanks for the quick support team!
Category: Inquiry

PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>

5. Analyze informal language handling.

PROMPT: Analyze how Zero-shot, One-shot, and Few-shot prompting handle informal language and slang in social media posts.

State which method performs best and why.

Analysis of Informal Language Handling

- **Zero-shot prompting:**

It can understand common informal words and slang but may misclassify posts when language is highly casual, abbreviated, or sarcastic due to lack of guidance.

- **One-shot prompting:**

With one example, the model gains some context about tone and intent, improving classification of informal posts, but accuracy is still limited for diverse slang usage.

- **Few-shot prompting:**

Multiple examples expose the model to different informal expressions, abbreviations, and tones, enabling better interpretation of slang and intent.

Best Performing Method:

Few-shot prompting performs best because repeated examples reduce ambiguity, help the model adapt to informal language patterns, and improve both accuracy and consistency in social media post classification.

